

Technical Problem Definition — Patent #4

Trust Continuity

Problem Statement (one sentence)

Computational trust cannot survive delegation.

When a trusted computation delegates work to another agent, system, or organization, the trust that existed in the original computation does not transfer to the delegated computation. The recipient has no verifiable connection to the trust that authorized, sourced, or scored the original decision. Trust breaks at the delegation boundary.

Why This Problem Exists

Delegation is fundamental to modern computing. Every multi-agent system, every microservice architecture, every cross-organizational workflow involves delegation — one entity assigning work to another. When the delegating entity has established trust (through verifiable entropy, quality scoring, cryptographic attestation), that trust is lost the moment work crosses a boundary.

Specifically:

The delegation boundary destroys three things:

1. **Provenance linkage.** The delegated agent receives a task but not the trust artifact that authorized it. There is no cryptographic link

between "this task was assigned" and "this is the verifiable entropy that determined the assignment."

2. **Quality continuity.** The original computation may have used AAA-grade entropy (ECS 950, three independent sources, blockchain-anchored). The delegated computation may use no verifiable entropy at all. There is no mechanism to ensure — or even detect — that trust quality degrades through delegation.

3. **Audit continuity.** An auditor examining the delegated computation sees a task that was performed. They cannot trace backward to the original trust assertion that authorized the delegation, verify the quality of that assertion, or confirm the delegation was derived from it.

The result: After delegation, trust is not reduced or degraded — it simply vanishes. The delegated computation operates in a trust vacuum, as if no trust infrastructure existed.

Why Existing Approaches Fail

Approach	What it provides	Why it fails for trust continuity
Task queues (Celery, SQS, Kafka)	Reliable task delivery	Deliver tasks, not trust. A message in a queue carries payload and routing, not provenance, quality scores, or cryptographic attestation of the decision that created the task.
API authentication (OAuth, JWT, mTLS)	Identity of the caller	Proves <i>who</i> delegated, not <i>why</i> the delegation was trustworthy. A valid OAuth token says "this agent is authorized to delegate." It does not say "the delegation decision was made using

Approach	What it provides	Why it fails for trust continuity
		quality-scored, verifiable entropy."
Distributed tracing (OpenTelemetry, Jaeger)	Request flow visibility	Traces the <i>execution path</i> (which services handled which requests). Does not trace the <i>trust path</i> (which trust artifacts backed which decisions). Tracing answers "what happened?" Trust continuity answers "was each step trustworthy?"
Blockchain event logs	Immutable record of events	Records <i>that</i> delegation occurred. Does not carry the trust artifact that <i>authorized</i> the delegation, the quality score of that artifact, or the cryptographic linkage between parent and child trust assertions.
PKI certificate chains	Identity hierarchy	Proves identity delegation (CA → intermediate → end entity). Does not prove computational trust delegation (this task was assigned based on this quality-scored entropy and can be traced back to this root trust source).

The common failure: Every existing approach handles delegation mechanics (delivery, authentication, tracing, logging). None handles **trust delegation** — ensuring that the trust properties (provenance, quality, attestation, verifiability) of the parent computation survive into the child computation.

Technical Effect (what solving this enables)

If trust can survive delegation:

1. **Multi-agent AI systems become auditable.** When Agent A delegates to Agent B based on verifiable entropy, and Agent B delegates to Agent C, an auditor can traverse the entire trust chain from C back to A, verifying every link independently.
2. **Cross-organizational computation becomes trustworthy.** When Enterprise X delegates a computation to Service Provider Y, the trust that Enterprise X established (quality-scored, signed, anchored) persists into Y's computation — without X and Y needing to trust each other.
3. **Trust degradation becomes visible.** Instead of trust vanishing at delegation boundaries, each delegation records a trust delta — exactly how much trust was preserved, lost, or transformed. Degradation is transparent, quantified, and auditable.
4. **Regulatory compliance extends through delegation chains.** A compliance framework requiring "all AI decisions must be backed by verifiable trust evidence" can be satisfied even when the decision involves multiple delegated agents across multiple organizations.

Invention Thesis (one sentence)

A system for preserving computational trust across delegation boundaries by cryptographically linking child trust assertions to parent trust assertions through lineage records that enforce non-amplification, record degradation transparently, and enable independent verification of the complete trust chain from any node to the root — without requiring cooperation from any intermediate node.

Pressure Test

Question	Answer
Does this problem exist today?	Yes. Every multi-agent AI deployment, every microservice delegation, every cross-org API call breaks trust. Today this is invisible because trust infrastructure itself is new. As #1–#3 make trust artifacts real, the delegation break becomes the next critical gap.
Does this exist without AI?	Yes. Any system that delegates computation — microservices, supply chains, federated systems, robotic fleets — faces trust continuity failure. AI makes it more urgent but doesn't make it exclusive.
Is it a technical problem, not a business problem?	Yes. The problem is the absence of a cryptographic linking mechanism between parent and child trust assertions. The solution is a protocol for trust-preserving delegation with provenance, quality continuity, and independent verification.
Can it generate many implementations?	Yes. Trust chains (DAG lineage), multi-party consensus, cross-boundary handoff packages, trust routing, threshold signatures, federated trust networks — all are specific implementations of the broader trust continuity problem.
Can you say it in one sentence?	"Computational trust cannot survive delegation." ✓

This document defines the technical problem. RFC-0002 will specify the protocol. Patent #4 claims will naturally derive from the protocol.